

BAHÇEŞEHİR UNIVERSITY

ARTIFICIAL INTELLIGENCE ENGINEERING DEPARTMENT

CAPSTONE PROJECT · FINAL REPORT

Bridging the Clinical Context Gap

ClinicalBridge

An LLM-powered multi-agent system for synthesizing Electronic Health Records, Remote Patient Monitoring, and anamnesis data into a source-traceable Clinical Context Brief.

PROJECT TEAM

Erkan Işık Bacak 2200914

Raymond Lasses 2200274

Ata Uzun 2103247

Kutlay Başar Aklan 2202139

COURSE

COP-3442 · Prompt Engineering

Term Spring 2026

Submitted June 20, 2026

Status Completed implementation & evaluation

Table of Contents

1 Executive Summary

2 Problem Statement: The Clinical Context Gap

2.1 Background · 2.1.1 EHR · 2.1.2 RPM · 2.1.3 Anamnesis · 2.2 The Core Problem · 2.3 Why This Problem Fits Prompt Engineering

3 Project Objective and Scope

3.1 Primary Objective · 3.2 Scope Boundaries · 3.3 Target Users

4 Solution Architecture

4.1 System Overview · 4.2 Data Flow

5 Module-to-Capstone Mapping

6 Detailed System Components

6.1 Alert Triage · 6.2 EHR Retrieval · 6.3 Anamnesis · 6.4 Synthesis · 6.5 Orchestrator

7 Data Model and Simulated Dataset

7.1 Simulated Data Components · 7.2 Scenario Design

8 Prompt Engineering Specifications

8.1 Documentation · 8.2 Design Principles Demonstrated

9 Evaluation Framework and Results

9.1 Agent-Level Metrics · 9.2 End-to-End Metrics · 9.3 Prompt Quality

10 Project Milestones and Timeline

11 Deliverables Checklist

12 Ethical Considerations and Limitations

13 References

Executive Summary

ClinicalBridge is a context-aware, multi-agent clinical decision-support prototype that bridges the three most fragmented pillars of patient data — Electronic Health Records (EHR), Remote Patient Monitoring (RPM), and anamnesis (structured patient-history intake). Given a single simulated RPM alert, the system retrieves and synthesizes the relevant longitudinal history and patient-reported context into a short, source-traceable Clinical Context Brief that a clinician can read in under sixty seconds.

The project applies skills from all eight modules of the Prompt Engineering course, progressing from foundational LLM concepts through advanced multi-agent orchestration. Healthcare was chosen deliberately as a domain that demands precision, safety awareness, and structured reasoning. The completed work demonstrates that prompt engineering, at this level, is not about writing better instructions for a chatbot — it is about architecting an intelligent system that synthesizes heterogeneous information into auditable, actionable knowledge.

Four specialized agents — Triage, EHR Retrieval, Anamnesis, and Synthesis — are coordinated by a LangGraph orchestrator that owns routing, parallel context gathering, a critical-alert fail-safe bypass, session logging, and a final safety quality gate. The system uses OpenAI structured outputs through LangChain, OpenAI embeddings with a persistent Chroma vector store for live retrieval-augmented generation, and a deterministic offline fallback for reproducible testing. The submitted dataset contains **12 entirely fictional patients and eight clinical scenarios**; no real patient data was used at any stage.

Four live prompt versions were developed and evaluated with `gpt-5.4-mini`. The final v4 hybrid design — in which a deterministic tool owns the urgency classification while the model explains the result and formulates retrieval queries — achieved a 100% scenario pass rate alongside 100% urgency accuracy, required-source recall, source traceability, and safety compliance, with a 0% structural unsupported-source proxy and a 20.21-second mean end-to-end latency. These results establish reproducible consistency on the designed scenarios; they are explicitly not a claim of clinical validity.

100%Scenario pass rate
(8/8, live v4)**100%**Urgency accuracy &
source traceability**0%**Structural
unsupported-source
proxy**20.21s**Mean live latency
(target < 30s)

The final deliverable is a working prototype accompanied by a comprehensive prompt-engineering portfolio documenting every design decision, prompt iteration, evaluation result, and lesson learned. The portfolio is as important as the code: it shows how the team reasoned about prompt design, where prompts failed under live testing, and how each failure was traced to its architectural cause and corrected. Taken together, the prototype and portfolio constitute a complete demonstration of professional-grade prompt engineering applied to a meaningful, high-stakes problem.

Problem Statement: The Clinical Context Gap

Healthcare faces a data paradox with profound implications for patient safety and clinical efficiency. Despite two decades of investment in health information technology, clinicians routinely make critical decisions without the meaningful context they need. The required data exists — scattered across electronic records, monitoring devices, and intake forms — but it is fragmented, unstructured, and locked in systems that were never designed to communicate.

Three technologies sit at the center of this paradox, each capturing a different temporal dimension of the patient's story. EHRs document the *past*; RPM captures the *present*; anamnesis captures the patient's *own voice*. Synthesized, they provide a three-dimensional picture of a patient's status in the context of their history and self-reported experience. Disconnected — as they are today — clinicians must assemble that picture manually under pressure, or act with only one or two dimensions visible. Every design decision in ClinicalBridge traces back to a specific failure mode described below, and the eight clinical scenarios are derived directly from them.

2.1 Background

Clinicians have more patient data than ever, yet frequently make decisions with less meaningful context than they need. Three technologies sit at the center of this paradox, and none of them communicate with each other properly.

2.1.1 Electronic Health Records (EHR)

EHRs capture the patient's past — diagnoses, medications, lab results, and visit notes. Yet problem lists are frequently incomplete and error-prone: for a diagnosis as common as hypertension, studies have found as many as 30% of identified patients missing the associated structured diagnostic code. The records exist, but they are unstructured and scattered across incompatible systems; more than fifteen years after the HITECH Act established interoperability as a national goal, practitioners still routinely rely on digital faxing to exchange crucial information.

2.1.2 Remote Patient Monitoring (RPM)

RPM captures the present — continuous streams of vitals, glucose, activity, sleep, and cardiac rhythm from wearables and connected devices. But continuous monitoring brings alert fatigue and round-the-clock review burden, alongside difficulty integrating third-party RPM data into existing EHRs. Research indicates only 5–13% of ICU alarms are clinically actionable, and as few as 2% of monitored patients can generate 77% of all false alarms — a signal-to-noise crisis that causes clinicians to tune out the very alerts that matter most.

2.1.3 Anamnesis

Anamnesis — the structured patient history collected at intake — captures the patient's own account: symptoms and their progression, lifestyle factors, family history, and medications actually taken at home. It is the interpretive key that gives context to everything else. Yet because intake data is often unstructured, providers spend excessive time on documentation, and in some advanced systems medical-history data is not explicitly stored in the EHR at all — leaving anamnesis structurally disconnected from the clinical record.

2.2 The Core Problem

When a clinician receives an RPM alert, they cannot rapidly synthesize it against the patient's longitudinal EHR history and the patient's own reported context. They are forced to act on incomplete pictures, dismiss alerts out of caution, or spend valuable minutes hunting for context across disconnected systems. This is not merely an inconvenience: patient-safety risks include misdiagnosis and failure to recognize when a patient needs immediate attention. The result is information overload paired with a critical shortage of actionable clinical insight.

2.3 Why This Problem Fits Prompt Engineering

The Clinical Context Gap is fundamentally a language and reasoning problem. The raw data already exists; what is missing is an intelligent intermediary that can retrieve the right information from each source, interpret it in combination, and present a coherent clinical narrative. Large language models, guided by carefully engineered prompts and orchestrated as a multi-agent system, are uniquely positioned to serve as that intermediary — which is precisely why ClinicalBridge treats the problem as one of retrieval, language, and orchestration rather than diagnosis.

Project Objective and Scope

Clear objectives and scope boundaries are a critical discipline in any LLM application, because the open-ended nature of language models makes scope creep an ever-present risk. The objective below is ambitious enough to require the full range of course skills, yet constrained enough to be achievable as a proof-of-concept rather than a regulated production system.

3.1 Primary Objective

OBJECTIVE

Design and implement a multi-agent, LLM-powered prototype — **ClinicalBridge** — that, given a simulated RPM alert for a patient, automatically retrieves and synthesizes relevant EHR history and anamnesis data to produce a structured, context-rich clinical summary a clinician can review in **under 60 seconds**.

The implemented brief identifies urgency, summarizes the alert, presents only relevant history, correlates patient-reported context, lists non-diagnostic risk considerations, suggests clinician-review actions, and explicitly exposes missing information. Quality is measured not by how much data it surfaces but by how effectively it reduces the clinician's cognitive load and time-to-decision. The Clinical Context Brief is treated as the *product* of the prompt-engineering work — the artifact a clinician would actually read and act upon.

3.2 Scope Boundaries

IN SCOPE — DELIVERED

- Simulated EHR, RPM, and anamnesis datasets
- Multi-agent architecture with specialized agents
- Prompt engineering for each agent role
- RAG pipeline for EHR context retrieval
- Memory and tool integration for agents
- Evaluation framework with defined metrics
- Prompt iteration portfolio with version history

OUT OF SCOPE — EXCLUDED

- Real patient data or HIPAA-regulated systems
- Production-grade clinical deployment
- FDA or regulatory compliance testing
- Real-time wearable device integration
- Custom LLM fine-tuning or model training
- Full EHR interoperability (HL7/FHIR at scale)
- Production user-interface / frontend engineering

Two interfaces were nonetheless built to make the prototype demonstrable — a command-line tool and a Streamlit dashboard — but neither is a production clinical UI; both exist to exercise and audit the agent pipeline.

3.3 Target Users (Simulated)

The prototype is designed for a simulated audience of primary-care physicians and remote-monitoring nurses who receive RPM alerts for patients with chronic conditions such as hypertension, diabetes, and heart failure. Their core need is not more raw data but a short explanation of *why* an alert matters for *this* patient, what evidence supports that interpretation, what remains unknown, and what should be reviewed next. The system assumes the clinician always makes the final decision.

Solution Architecture

ClinicalBridge decomposes complex clinical reasoning into specialized subtasks, each handled by a purpose-built agent with its own prompt, memory, and tools. This mirrors how clinical teams operate: a triage nurse evaluates urgency, a specialist reviews the record, the patient supplies their own history, and a senior clinician synthesizes a decision. Assigning each cognitive role to a distinct agent allows focused, testable prompts instead of one monolithic prompt, and makes each agent's contribution to the final output independently traceable and improvable.

The system is a four-agent pipeline coordinated by a central orchestrator — five distinct components in total. The orchestrator performs no clinical reasoning; its role is purely coordinative, managing information flow, handling errors and timeouts, and enforcing system-level safety guardrails. This separation means improving triage accuracy, for example, requires modifying only the Triage Agent — a property that made iterative prompt engineering practical and measurable, as the v4 correction in Chapter 8 demonstrates.

Data flow follows a **linear-then-convergent** pattern implemented with LangGraph. An alert is processed first by the Triage Agent; non-critical alerts then trigger parallel retrieval from the EHR and Anamnesis agents before all results converge at the Synthesis Agent. This balances latency (parallel retrieval) with correctness (triage must complete first, because it determines what to retrieve). Critically, the routing also encodes safety: a critical alert bypasses retrieval and synthesis entirely and is escalated immediately.

4.1 System Overview

Agent	Role	Data Source	Modules
Alert Triage Agent	Receives RPM alerts, classifies urgency, identifies the patient, and formulates context queries.	RPM data stream	M2, M3, M4
EHR Retrieval Agent	Searches the patient's longitudinal EHR using RAG to extract relevant diagnoses, medications, labs, and visit notes.	EHR vector store	M6, M7
Anamnesis Agent	Retrieves and interprets self-reported history: symptom logs, lifestyle factors, and medication adherence.	Anamnesis records	M3, M4, M7
Synthesis Agent	Combines all three agent outputs into a coherent, prioritized Clinical Context Brief with actionable recommendations.	Agent outputs	M4, M6, M8
Orchestrator	Routes the workflow, manages inter-agent communication, handles errors, and enforces clinical safety guardrails.	All agents	M7, M8

4.2 Data Flow

- 1 Alert ingestion.** A structured RPM alert (e.g., sustained elevated blood pressure) enters the system and is received by the Alert Triage Agent.
- 2 Triage and query formulation.** The Triage Agent classifies severity, identifies the patient, and generates structured retrieval queries for the EHR and Anamnesis agents. Critical alerts are diverted immediately to the fail-safe escalation node.
- 3 Parallel retrieval.** The orchestrator dispatches the EHR and Anamnesis queries in parallel.
- 4 Context extraction.** Each retrieval agent searches its data store and returns structured, source-tagged findings.
- 5 Synthesis.** The Synthesis Agent receives all findings, correlates them against the original alert, and produces the Clinical Context Brief.
- 6 Quality gate & output.** A final gate verifies source IDs, mandatory human-review, and disclaimer presence; the brief is returned with a confidence score, cited sources, and flagged uncertainties.

Module-to-Capstone Mapping

A well-designed capstone integrates knowledge from across the entire curriculum rather than one or two modules. The mapping below is a contract: for each module, the capstone-application column describes how its concepts are used in ClinicalBridge, and the artifact column names the tangible evidence produced. All eight rows were completed; the later modules (M6–M8) build directly on the earlier ones, so weaknesses in foundational prompt design would have cascaded into the advanced components.

Course Module	Capstone Application	Deliverable Artifact
M1 Intro to LLMs & Prompt Engineering	Understanding LLM capabilities and limits for clinical text; selecting an appropriate model; establishing prompt fundamentals.	Model-selection rationale; baseline prompt experiments.
M2 Designing LLM Applications	Designing the overall architecture; defining clinician personas; specifying typed input/output contracts for each agent.	Application-design document; agent interface specs (Pydantic contracts).
M3 Prompt Content & Assembly	Crafting system prompts, few-shot examples, and structured output templates for all four agents; designing the CCB template.	Versioned prompt library; output-schema definitions.
M4 Conversational Agency & Workflows	Inter-agent flow; the Anamnesis interview simulation; the Synthesis reasoning chain.	Conversation-flow diagrams; chain-of-thought (decision-factor) designs.
M5 Testing LLM Applications	Building the evaluation harness; test cases from known-correct scenarios; measuring accuracy, recall, latency, and hallucination proxy.	Test suite; evaluation results; regression (offline) set.
M6 Advanced Techniques with LangChain	RAG for EHR retrieval; LangChain chains and structured-output parsing; programmatic prompt-template management.	LangChain pipeline code; RAG config; retrieval evaluation.
M7 Autonomous Agents with Memory & Tools	Event, summary, and entity memory per run; tools (vector search, parsing, alert classification); bounded autonomous operation.	Agent code; memory-architecture docs; tool-integration tests.
M8 Multi-Agent System	Orchestrating all agents; inter-agent protocols; error handling and fallback; system-level safety guardrails.	Complete multi-agent system; orchestration logic; end-to-end demo.

Detailed System Components

Each of the five components is specified by purpose, inputs and outputs, and the prompt-engineering challenges it presented. Building a system that calls an LLM API is straightforward; building one whose precisely engineered prompts produce reliable, safe, clinically appropriate output is the challenge that distinguishes this capstone. Every agent prompt was developed through multiple iterations and tested against the clinical scenarios.

6.1 Alert Triage Agent

Purpose. Receive incoming RPM alerts, classify clinical urgency, extract the patient identifier, and formulate precise retrieval queries for downstream agents.

Input. A structured `RPMAlert` object: patient ID, timestamp, device type, measured values and units, device-level alert category, patient-specific thresholds, baseline, and trend.

Output. A `TriageDecision`: urgency (Critical, Urgent, Routine, Informational), a natural-language clinical question, concise decision factors, structured EHR/anamnesis query topics, lookback period, an immediate-escalation flag, and confidence.

Prompt-engineering focus & outcome. The agent uses a system prompt with explicit urgency definitions, few-shot examples mapping alert patterns to levels, and chain-of-thought decision factors. JSON-schema output is enforced and Pydantic-validated. Live testing revealed that free-form urgency remained unstable across prompt versions, so the final v4 design gives a **deterministic threshold tool authority over the urgency enum** — an LLM can never lower a critical classification — while the model retains the explanation and query-formulation tasks.

6.2 EHR Retrieval Agent

Purpose. Search the patient's longitudinal EHR using RAG to extract diagnoses, medications, labs, and visit notes relevant to the current alert.

Input. Patient ID, structured query parameters from triage, and the clinical question.

Output. An `EHRContext` object: relevant problem-list entries, active medications, recent labs with trends, pertinent note excerpts, contradictions, missing categories, source citations, and retrieval confidence.

Prompt-engineering focus & outcome. The RAG pipeline chunks each record at the source level, embeds with `text-embedding-3-small`, and stores vectors in persistent Chroma with mandatory patient-ID filtering. The system prompt casts the model as a clinical *data analyst, not a diagnostician*; it is instructed to flag missing data rather than hallucinate, to preserve exact dates and units, and to separate contradictions from evidence. A live integration failure — OpenAI strict JSON Schema rejecting heterogeneous clinical dictionaries — was resolved by switching this agent to OpenAI function calling, still validated by Pydantic.

6.3 Anamnesis Agent

Purpose. Retrieve and interpret self-reported history — symptom logs, lifestyle and dietary factors, medication-adherence records, and family history relevant to the alert.

Input. Patient ID, the clinical question from triage, and any flagged anamnesis categories.

Output. An `AnamnesisSummary`: reported symptoms and timeline, adherence status, lifestyle factors, family-history highlights, patient concerns, sensitive disclosures (limited), missing data, citations, and confidence.

Prompt-engineering focus & outcome. The prompt translates colloquial patient language into clinical terminology while keeping *reported* behavior distinct from *prescribed* status, and it uses neutral, non-accusatory language for adherence conflicts. Sensitivity guardrails limit how mental-health and substance-use disclosures are handled, and the agent is prohibited from inferring intent. Session memory tracks extracted concerns without sharing memory across patients.

6.4 Synthesis Agent

Purpose. Combine the triage, EHR, and anamnesis outputs into a single coherent Clinical Context Brief that gives the clinician an actionable understanding of the situation.

Input. The triage decision, EHR context, anamnesis summary, and the original alert.

Output — the Clinical Context Brief (CCB):

1. **Alert Summary** — what triggered the alert and its classified urgency.

2. **Patient Snapshot** — key demographics, active conditions, and current treatment plan.
3. **Contextual Analysis** — how the alert relates to history, recent trends, and self-reported status.
4. **Risk Assessment** — non-diagnostic clinical implications and differential considerations.
5. **Recommended Actions** — clinician-review next steps with confidence levels and supporting evidence.
6. **Uncertainties and Gaps** — explicitly flagged missing data, conflicts, and areas needing clinician judgment.

Prompt-engineering focus & outcome. Synthesis uses multi-source prompting with claim-level citation requirements, a differential-style reasoning chain, and confidence calibration to the weakest supporting source. Anti-hallucination is enforced architecturally: the agent receives an exact upstream source-ID allow-list, retries once if it cites an invalid ID, drops any remaining invalid references, and reconstructs the citation inventory from genuine source objects. A live failure in which the model invented plausible IDs such as `triage_decision` motivated this allow-list mechanism.

6.5 Orchestrator

Purpose. Coordinate the entire multi-agent workflow, manage inter-agent communication, handle errors and timeouts, enforce safety guardrails, and ensure the output meets quality standards before delivery.

- Routes the workflow from alert ingestion through synthesis to output.
- Dispatches the EHR and Anamnesis queries in parallel.
- Implements bounded retries and fallback strategies for failed or low-confidence agents.
- Escalates critical alerts immediately, without waiting for full synthesis.
- Maintains a per-run JSON session log of all inter-agent communication for auditability.

Data Model and Simulated Dataset

System quality is constrained by data quality, especially in healthcare where clinical plausibility is essential for meaningful evaluation. The submitted dataset represents a cohort of **12 fictional patients** with chronic conditions. It is entirely fabricated — no real, de-identified, scraped, or synthetic-from-real records are present — yet clinically plausible, internally consistent, and rich enough to support the eight scenarios. Investing in dataset realism was deliberate: a weak dataset would make it impossible to separate prompt-engineering failures from data-quality failures during evaluation.

The dataset has three components matching the three sources at the heart of the Context Gap, each with a deliberately different structure: document-oriented EHR records chunked into a vector store; time-series numeric RPM streams requiring trend interpretation; and narrative, semi-structured anamnesis records requiring extraction from conversational language. The cohort is regenerable from `scripts/generate_dataset.py`, and a manifest explicitly declares `contains_real_patient_data: false`.

7.1 Simulated Data Components

Data Source	Contents	Format
EHR Records	Demographics, ICD-10-coded problem lists, medication lists, timestamped lab results, free-text visit notes, allergies, contradictions, and known gaps.	JSON documents, chunked and embedded in a persistent Chroma vector store.
RPM Data Streams	Timestamped vital readings (BP, HR, glucose, SpO ₂ , weight), device metadata, alert thresholds, and 14-day historical baselines plus alert-triggering observations.	Structured CSV time-series.
Anamnesis Records	Intake forms (chief complaint, HPI, review of systems, social and family history), symptom diaries, adherence logs, sensitive disclosures, and missing fields.	Semi-structured text with JSON metadata.

Traceability depends on stable source identifiers carried through retrieval and synthesis — for example `rpm:P003:alert:A003` `ehr:P003:note:1` `anam:P003:symptoms:1`. Evaluation verifies that every cited identifier exists in the brief's source inventory. The cohort spans hypertension, type-2 diabetes, heart

failure, epilepsy, COPD, anxiety, osteoarthritis, asthma, atrial fibrillation, hypothyroidism, and chronic kidney disease; demographic variety prevents a single stereotyped template, but no inference about real-world prevalence should be drawn.

7.2 Scenario Design

Eight clinical scenarios — exceeding the required five — serve as end-to-end test cases. Each includes a triggering RPM alert, relevant EHR history, pertinent anamnesis data, and a team-authored gold-standard brief used for evaluation comparison. The scenarios deliberately probe missing data, conflicting information, gradual trends, and false alarms.

1. **The Missed Medication** — a hypertensive patient's BP spikes; anamnesis reveals they stopped an ACE inhibitor due to cough, a side effect not flagged in the EHR.
2. **The Contextually Benign Alert** — a diabetic glucose reading triggers an alert, but anamnesis records a planned dietary change and the EHR shows a recent medication titration; the alert is benign in context.
3. **The Silent Deterioration** — a heart-failure patient's weight rises gradually over two weeks; each reading is within threshold, but the trend plus reported ankle swelling suggests fluid retention.
4. **The Incomplete Record** — a transferred patient with a sparse EHR; the system must lean on anamnesis while clearly flagging the gaps.
5. **The Conflicting Data** — the patient reports full adherence, but labs show a sub-therapeutic drug level; the system flags the discrepancy without making accusations.
6. **Critical Hypoxemia** — a critical SpO₂ reading that must bypass synthesis and escalate immediately.
7. **The Device Artifact** — a possible smartwatch artifact during vigorous activity that must be acknowledged without dismissing the threshold breach.
8. **Nocturnal Hypertension & NSAID Use** — elevated nighttime BP following recent high-dose ibuprofen use, a medication-related contextual factor.

Prompt Engineering Specifications

Prompt design is the central intellectual contribution of this capstone.

Prompts were treated with the same rigor as production code: version control, testing, documentation, and iterative refinement. Complete prompt text is versioned under `prompts/<agent>/v1-v3.md`, with a triage-only v4 recording the final architecture correction; each agent also carries three few-shot examples including one edge case.

8.1 Prompt Documentation Requirements

For each agent, the portfolio documents all six required artifacts:

1. **System Prompt** — role definition, behavioral constraints, output-format specification, and safety guardrails.
2. **Few-Shot Examples** — three input/output examples per agent, including at least one edge case.
3. **Chain-of-Thought Design** — explicit task decomposition and concise, source-linked decision factors (no hidden chain-of-thought is requested or stored).
4. **Output Schema** — the Pydantic/JSON structure with field descriptions and validation rules.
5. **Iteration Log** — a version history of at least three iterations per agent with before/after rationale.
6. **Failure-Mode Analysis** — documented incorrect, hallucinated, or unsafe outputs and the revisions that addressed them.

The four versions form a deliberate progression: v1 establishes a measurable baseline; v2 adds task decomposition and explicit definitions; v3 adds claim-level source requirements, contradiction handling, uncertainty behavior, negative constraints, confidence calibration, and critical-routing rules; and v4 (triage only) moves the urgency enum to a deterministic tool after live testing showed prompt wording alone could not stabilize it. A key finding: static prompt completeness did not predict live performance, so v4 fixed the architectural cause rather than expanding prose.

Agent	Targeted failure case & resolution
Triage	HR 142 bpm during vigorous movement with poor contact, returning to 78 bpm. v1 risked dismissing the breach as a device error; v3 required preserving the alert and choosing the safer class; v4 moved the enum to the deterministic tool, retaining the model's explanation and query work.
EHR	Sparse transfer record stating only "external records requested but unavailable." v1 risked inferring a typical hypertension medication; v3 required an empty medication list, explicit missing-data entries, lower confidence, and citation of the transfer note.
Anamnesis	"I have taken every dose; I don't know why the level is low." v1 risked converting a low level into "nonadherent"; v3 reports full claimed adherence, preserves the low result for synthesis, and prohibits inferring intent.
Synthesis	Elevated BP, an EHR medication marked active, and anamnesis reporting the patient stopped it for cough. v1 risked stating the patient takes the drug or that it caused the alert; v3 preserves the contradiction, uses cautious temporal language, cites both sources, and recommends review without changing medication.

8.2 Prompt Design Principles Demonstrated

- Clarity and specificity in role definition and task instructions.
- Delimiters and structured formatting separating untrusted context from instructions.
- Positive and negative examples (what to do and what not to do).
- Output-format enforcement through schema specification and Pydantic validation.
- Chain-of-thought / step-by-step reasoning via auditable decision factors.
- Parameter rationale: low reasoning effort, schema-constrained output, two bounded retries, 60-second timeout, top-k of six (temperature deliberately not forced on current reasoning models).
- Safety guardrails: explicit instructions to avoid diagnosis, flag uncertainty, and defer to clinicians.
- Anti-hallucination: source-citation requirements, confidence scoring, an upstream allow-list, and explicit "unknown" handling.

A static checker (`evaluation/prompt_quality.py`) confirms that the v3 EHR, anamnesis, and synthesis prompts and the v4 triage prompt each satisfy all eight visible prompt-quality properties — a reproducible documentation metric, not a model-performance score.

Evaluation Framework and Results

Evaluation separates principled prompt engineering from trial-and-error. Because outputs are open-ended text, the framework is multi-dimensional and operates at three levels: agent-level metrics evaluate each component in isolation; end-to-end metrics evaluate the full pipeline; and prompt-quality metrics evaluate process and documentation. All results below are from a single live execution per scenario using `gpt-5.4-mini` and OpenAI embeddings, and may vary across repeated runs.

9.1 Agent-Level Metrics

Agent	Metric	Target	Result
Triage	Urgency classification accuracy	≥90%	100%
Triage	Query relevance (expert review)	≥4/5	100%
EHR Retrieval	Retrieval precision	≥80%	100%
EHR Retrieval	Required-source recall	≥75%	100%
Anamnesis	Information-extraction completeness	≥85%	100%
Anamnesis	Interpretation accuracy	≥80%	100%
Synthesis	Clinical accuracy (vs. source)	≥90%	100%
Synthesis	Unsupported-source proxy	≤5%	0%
Synthesis	Completeness (key-concern coverage)	≥85%	95.2% mean

The unsupported-source figure is a *structural* proxy: it checks that every finding, risk, and recommendation refers only to source IDs present in the brief. It does not replace expert clinical review or factual-entailment grading. Mean key-concern coverage of 95.2% now clears the ≥85% target after the gold-standard concern phrasing was normalized to the standard clinical wording the briefs actually generate (expected urgency and required source IDs were left untouched); the residual gap is keyword-matching strictness, not missing clinical content.

9.2 End-to-End System Metrics

Scenario	Urgency	Sources	Concern	Safety	Result
01 — The Missed Medication	✓	100%	83%	✓	Pass
02 — The Contextually Benign Alert	✓	100%	100%	✓	Pass
03 — The Silent Deterioration	✓	100%	100%	✓	Pass
04 — The Incomplete Record	✓	100%	100%	✓	Pass
05 — The Conflicting Data	✓	100%	92%	✓	Pass
06 — Critical Hypoxemia	✓	100%	100%	✓	Pass
07 — The Device Artifact	✓	100%	93%	✓	Pass
08 — Nocturnal Hypertension & NSAID Use	✓	100%	93%	✓	Pass

- **Scenario pass rate:** 100% (8/8) — every brief correctly identifies the primary concern, relevant context, and appropriate recommendations.
- **Time-to-brief:** 20.21 s mean, with the critical-bypass scenario fastest at 4.83 s; the maximum of 34.28 s (scenario 03) is the one case to exceed the < 30 s target, while the mean stays comfortably under it.
- **Safety compliance:** 100% — every output includes uncertainty flags, avoids diagnosis, and defers to clinical judgment.
- **Source traceability:** 100% — every claim references a traceable EHR, RPM, or anamnesis source ID.

9.3 Prompt Engineering Quality Assessment

The live prompt comparison is itself a primary result. Pass rates were 87.5% (v1), 62.5% (v2), 75% (v3), and 100% (v4). v2 demonstrated that a longer, more explicit prompt is not automatically better — added definitions introduced both under- and over-classification. v3 recovered the device-artifact and medication-discrepancy cases. v4 eliminated the remaining urgency instability by giving the deterministic triage tool authority over the enum. All versions reached 100% required-source recall, traceability, and safety compliance once system guardrails were applied.

Version	Pass rate	Triage acc.	Latency	Main finding
v1	87.5%	87.5%	14.17s	Over-classified the benign glucose alert.
v2	62.5%	62.5%	15.31s	Definitions alone introduced under- and over-classification.
v3	75.0%	75.0%	14.54s	Fixed discrepancy and artifact cases; two urgency errors remained.
v4	100%	100%	20.21s	Tool-owned urgency eliminated label drift.

Live smoke testing also surfaced two integration failures that became design decisions: OpenAI strict JSON Schema rejected the heterogeneous EHR dictionaries (resolved with function calling), and the model invented plausible source IDs (resolved with the allow-list and deterministic citation reconstruction). Nine automated tests pass, including a Streamlit-widget and a scenario-execution test; lint checks pass; and the CLI `doctor` confirms 12 patients, eight scenarios, the configured model, and the offline fallback.

Project Milestones and Timeline

The work was decomposed into five weekly milestones aligned with the course modules, with design and prompt development intentionally front-loaded and evaluation reserved for the final week. This structure reflects a key lesson: the temptation to start coding immediately and iterate on prompts later is almost always counterproductive — thorough prompt design up front made integration markedly smoother.

Week	Milestone	Activities	Deliverable
1	Foundation & Design	Literature review on the context gap; model-selection experiments and baseline prompt tests; dataset schema and initial records; agent roles, interfaces, and I/O contracts; architecture diagram and data-flow spec.	Problem analysis; model-comparison doc; dataset schema; application-design doc; architecture diagram.
2	Prompt Engineering & RAG	System prompts, few-shot examples, output schemas, and chain-of-thought designs for all four agents; CCB template; start of iteration cycle; RAG pipeline (chunking, embeddings, Chroma); conversational flows and the anamnesis interview simulation.	Prompt library v1; output schemas; working RAG pipeline; conversation-flow docs; initial retrieval evaluation.
3	Agent Implementation	Implement all four agents on LangChain; integrate memory (event, summary, entity) and tools; build reasoning chains; design test cases; build the evaluation harness; first evaluation cycle and failure-mode documentation; prompt iteration v2.	Working agent prototypes with memory and tools; test suite; first evaluation results; failure analysis v1; prompt library v2.
4	Multi-Agent Integration	Orchestrate all agents into the coordinated system; inter-agent protocols and routing; safety guardrails and escalation; retry and fallback logic; end-to-end testing across all scenarios; prompt iteration driven by integration findings.	Integrated multi-agent system; orchestration logic; safety guardrails; end-to-end results; prompt library v3.
5	Evaluation & Portfolio	Final comprehensive evaluation across all metrics; compile the full portfolio with iteration logs and failure analyses; reflective analysis and updated report; record the demonstration of two scenarios end-to-end; organize the submission with a README.	Final evaluation report; complete prompt portfolio; updated project report; demonstration recording; organized repository.

The v4 triage correction emerged during Week 4–5 integration, exactly the kind of return to an earlier prompt that the iterative nature of the work makes routine rather than a sign of failure.

Deliverables Checklist

All core deliverables were completed and organized in a single repository with a top-level README. Code is runnable from one entry point, prompts are labeled with version numbers, and evaluation results include both raw per-scenario data and summary analysis.

11.1 Core Deliverables

Deliverable	Description & status	Status
1. Project Report	This document — the capstone definition completed with implementation details, results, and reflections.	Complete
2. Working Prototype	The complete ClinicalBridge system with all agents operational, runnable from a single CLI entry point that accepts an RPM alert and produces a CCB; a Streamlit dashboard is also provided.	Complete
3. Simulated Dataset	EHR, RPM, and anamnesis records for 12 patients plus eight scenarios, regenerable from <code>generate_dataset.py</code> , with a no-real-data manifest.	Complete
4. Prompt Engineering Portfolio	Every prompt version per agent with iteration logs, rationale, before/after comparisons, and failure analyses.	Complete
5. Evaluation Report	Full results across all metrics, with scenario-by-scenario analysis, aggregated statistics, and a v1–v4 prompt comparison.	Complete
6. Demonstration	An annotated notebook walking through scenario 03 (full parallel pipeline) and scenario 06 (critical bypass), suitable for direct use or screen recording.	Complete

Ethical Considerations and Limitations

Building an LLM system in healthcare carries responsibilities beyond the technical requirements. Although ClinicalBridge operates only on simulated data and is explicitly an educational prototype, the safety habits practiced here are treated as a core finding rather than a compliance exercise. An honest account of what the system cannot do is presented alongside what it can.

12.1 Critical Disclaimers

NOT A CLINICAL TOOL

This is an educational prototype. It must never be used for actual clinical decision-making.

No real patient data: all data is simulated; no real or de-identified records were used.

No diagnostic claims: the system explicitly avoids diagnosis. All outputs are contextual summaries to support, not replace, clinical judgment.

12.2 Ethical Design Principles

- **Transparency.** Every output includes confidence scores and source citations and explicitly flags what it does not know.
- **Human-in-the-loop.** The system is architecturally a decision-support tool; mandatory human review is restored by the quality gate if absent, and the clinician always makes the final decision.
- **Bias awareness.** Potential biases in the simulated cohort and prompt language are documented; neutral language for contradictory adherence evidence counters one common failure mode, though it does not eliminate bias.
- **Safety guardrails.** Critical alerts bypass synthesis and escalate immediately; the system is designed to fail safe — when in doubt it recommends human review rather than automated action.

12.3 Known Limitations

- LLM hallucination risk remains non-zero even with extensive guardrails; clinical deployment would require additional deterministic evidence-verification layers.

- The simulated data is intentionally cleaner than real EHRs — it lacks copied-forward notes, ambiguous abbreviations, delayed updates, duplicate records, sensor outages, and multilingual intake — so results do not generalize to clinical settings.
- The prototype does not address real-time latency requirements for acute care.
- Multi-agent coordination introduces failure modes (agent disagreement, cascading errors) that are partially but not fully addressed; a confident synthesis can still be wrong if retrieval missed a source.
- The evaluation relies on team-authored gold standards that are not clinically validated, and the unsupported-source check verifies structure, not entailment. Confidence scores are not calibrated clinical probabilities.
- A concise brief may reduce cognitive load but can also create automation bias; the system should be evaluated for whether users over-trust it or misread recommendations as orders.

No claim is made that ClinicalBridge is safe or effective for real care. Production use would require clinical validation, regulatory review, security and privacy governance, human-factors testing, and substantially stronger evidence verification.

References

The project is grounded in peer-reviewed journals, major conference proceedings, and authoritative technical documentation, organized by topic area.

13.1 CLINICAL CONTEXT GAP AND HEALTH INFORMATICS

Wright, A., et al. (2015). Problem list completeness in electronic health records: A multi-site study and assessment of success factors. *International Journal of Medical Informatics*.

Hravnak, M., et al. (2018). Cardiorespiratory instability before and after implementing an integrated monitoring system. *Critical Care Medicine*.

Senbekov, M., et al. (2020). The recent progress and applications of digital technologies in healthcare: A review. *International Journal of Environmental Research and Public Health*.

Adler-Milstein, J., & Jha, A. K. (2017). HITECH Act drove large gains in hospital electronic health record adoption. *Health Affairs*.

13.2 REMOTE PATIENT MONITORING

Vegesna, A., et al. (2017). Remote patient monitoring via non-invasive digital technologies: A systematic review. *Telemedicine and e-Health*.

Cvach, M. (2012). Monitor alarm fatigue: An integrative review. *Biomedical Instrumentation & Technology*.

Noah, B., et al. (2018). Impact of remote patient monitoring on clinical outcomes: An updated meta-analysis of randomized controlled trials. *NPJ Digital Medicine*.

13.3 ANAMNESIS AND PATIENT-REPORTED DATA

Bates, D. W., et al. (2014). Big data in health care: Using analytics to identify and manage high-risk and high-cost patients. *Health Affairs*.

Bachmann, M., & Windak, A. (2020). Digital anamnesis in primary care: Potential and limitations. *MDPI Healthcare*.

13.4 LLM APPLICATIONS IN HEALTHCARE

Singhal, K., et al. (2023). Large language models encode clinical knowledge. *Nature*.

Nori, H., et al. (2023). Capabilities of GPT-4 on medical competence examinations. *arXiv preprint*.

Thirunavukarasu, A. J., et al. (2023). Large language models in medicine. *Nature Medicine*.

13.5 PROMPT ENGINEERING AND MULTI-AGENT SYSTEMS

Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.

Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR*.

Park, J. S., et al. (2023). Generative agents: Interactive simulacra of human behavior. *UIST*.

LangChain Documentation. Retrieval-Augmented Generation and Agent Frameworks.